

Arquitectura de Computadores - Resumen de clases

Clase presencial 21 de abril + clase virtual 28 de abril de 2026

Cómo leer este resumen: La clase del 21 de abril se resume a partir del PDF presencial. La clase del 28 de abril se resume a partir del video, donde se retomó la representación de números fraccionarios, precisión y redondeo.

1. Ultra resumen para el trabajo

- El tema central es la representación de números reales/fraccionarios en máquina, especialmente IEEE 754 y el cambio de base de la parte fraccionaria.
- Un número flotante no representa necesariamente el valor real exacto: normalmente guarda una aproximación. Por eso aparece un error de representación.
- IEEE 754 divide el flotante en signo, exponente con sesgo y mantisa/fracción. También define casos especiales: ± 0 , $\pm$ infinito y NaN.
- Para convertir una fracción decimal $0 < r < 1$ a base b , se multiplica repetidamente por b , se toma la parte entera como dígito y se continúa con la nueva parte fraccionaria.
- El error al cortar una representación después de n dígitos queda acotado aproximadamente por $b^{(-n)}$. Esto permite calcular cuántos dígitos hacen falta para una precisión deseada.
- IEEE 754 permite modos de redondeo: al más cercano, hacia cero, hacia $+\infty$ y hacia $-\infty$. Estos modos importan cuando se acumulan muchísimas operaciones de punto flotante.

2. Clase presencial - 21 de abril: IEEE 754 y tipos flotantes

La clase trabaja la idea de precisión en punto flotante: el valor real x no siempre se almacena exactamente; se almacena una aproximación x_{barra} y un error ϵ .

$$x = x_{\text{barra}} + \epsilon$$

Si la aproximación queda por debajo del valor real, hay error por defecto. Si queda por encima, hay error por exceso. Al operar flotantes, los errores también se combinan; por ejemplo, al sumar dos aproximaciones, el error total incluye el error de cada operando.

$$a = a_{\text{barra}} + \epsilon_a$$

$$b = b_{\text{barra}} + \epsilon_b$$

$$a + b = (a_{\text{barra}} + b_{\text{barra}}) + (\epsilon_a + \epsilon_b)$$

2.1. Formatos de flotante vistos en clase

Tipo	Longitud total	Bits de exponente	Sesgo / bias
Simple	32 bits	8	127
Double	64 bits	11	1023
Quadruple	128 bits	15	16383
Octuple	256 bits	15	16383, según los apuntes de clase

El sesgo se calcula con:

$$\text{sesgo} = 2^{(\text{longitud_exponente} - 1)} - 1$$

Ejemplo: si el exponente tiene 8 bits, $\text{sesgo} = 2^{(8-1)} - 1 = 127$.

2.2. Estructura general de un flotante

La representación se puede pensar como tres campos:

- Signo: indica si el número es positivo o negativo.
- Exponente: no se guarda directamente; se guarda con sesgo para poder representar exponentes positivos y negativos.
- Mantisa/fracción: guarda los bits significativos del número, normalmente interpretados como 1.fracción en números normalizados.

$$\text{valor normalizado} = (-1)^{\text{signo}} * (1.\text{fracción}) * 2^{(\text{exponente_guardado} - \text{sesgo})}$$

2.3. Números especiales

- Cero: aparece cuando el exponente y la mantisa están en cero. Debido al bit de signo, existen +0 y -0.
- Infinito: representa desbordamientos o resultados que tienden a infinito dentro del modelo. En IEEE 754 estándar: exponente todo en 1 y mantisa en 0.
- NaN (Not a Number): representa resultados indefinidos o no numéricos. En IEEE 754 estándar: exponente todo en 1 y mantisa distinta de 0.
- Underflow: ocurre cuando el resultado es demasiado pequeño para representarse normalmente. Puede llevarse a 0, redondearse o manejarse con números subnormales/excepciones, según configuración.

Ojo: En los apuntes aparece una frase confusa sobre infinitos y mantisa. Para efectos prácticos de IEEE 754, recuerda: infinito = exponente todo 1 y mantisa 0; NaN = exponente todo 1 y mantisa no cero.

2.4. Idea importante: decimal finito no implica binario finito

Un número como 1/10 tiene expansión finita en base 10, pero expansión infinita en base 2. Por eso valores aparentemente simples como 0.1 no se guardan exactamente en binario.

Conclusión práctica: los errores no son fallos del programa necesariamente; son consecuencia de la representación finita en máquina.

3. Clase virtual - 28 de abril: cambio de base de fracciones y redondeo

La clase virtual continúa el tema de flotantes leyendo y comentando un texto sobre un teorema para cambiar de base números con parte fraccionaria. La idea central es justificar formalmente el algoritmo típico para convertir una fracción decimal a binario, hexadecimal u otra base.

3.1. Problema que se quiere resolver

Dado un número real r en el intervalo abierto $(0,1)$, se quiere escribirlo en base b como:

$$r = (0.a_1 a_2 a_3 \dots)_b$$

Eso significa que el valor real se expresa como una serie:

$$r = a_1 * b^{(-1)} + a_2 * b^{(-2)} + a_3 * b^{(-3)} + \dots$$

Cada dígito a_i debe ser un entero válido de la base:

$$0 \leq a_i < b$$

3.2. Algoritmo constructivo para obtener los dígitos

El método práctico que justifica el teorema es el siguiente:

1. Multiplicar la fracción r por la base destino b .
2. Tomar la parte entera del resultado: ese es el primer dígito a_1 .
3. Tomar la parte fraccionaria restante y volver a multiplicarla por b .
4. Repetir el proceso para obtener $a_2, a_3, a_4, \dots$
5. Si la fracción llega a 0, la expansión terminó. Si no, se obtiene una expansión infinita o periódica.

```
p = r*b
a1 = parte_entera(p)
q = p - a1
```

```
p = q*b
a2 = parte_entera(p)
q = p - a2
...
```

3.3. La función parte entera

La demostración se apoya en la función parte entera: para un real positivo x , $[x]$ es el mayor entero i tal que:

$$i \leq x < i + 1$$

La clase recalca que esta parte entera existe y es única. Esa unicidad es lo que permite afirmar que cada dígito de la representación también queda determinado de forma única.

3.4. Cota de error al cortar la expansión

Si se usan solo los primeros n dígitos de la representación, se obtiene una suma parcial S_n . El teorema permite acotar el error:

$$|r - S_n| < b^{-(n)}$$

Esta desigualdad es probablemente lo más útil para ejercicios: si quieres una precisión epsilon, necesitas elegir n para que $b^{-(n)}$ sea menor que epsilon.

$$b^{-(n)} < \epsilon \\ \Rightarrow n > \log_b(1/\epsilon)$$

3.5. Ejemplo comentado: convertir 0.1415 decimal a hexadecimal

En el video se muestra un ejemplo con base 16. El procedimiento produce estos primeros dígitos:

Paso	$p = \text{fracción} * 16$	Dígito hexadecimal	Nueva fracción
1	$0.1415 * 16 = 2.264$	2	0.264
2	$0.264 * 16 = 4.224$	4	0.224
3	$0.224 * 16 = 3.584$	3	0.584

4	$0.584 * 16 = 9.344$	9	0.344
5	$0.344 * 16 = 5.504$	5	0.504

Por tanto:

$$0.1415_{10} \approx 0.24395_{16}$$

Con tres o cuatro dígitos hexadecimales ya se logra una aproximación cercana si la tolerancia fuera de alrededor de 0.001, porque el error se controla con $16^{(-n)}$.

3.6. Situación inversa: si ya tengo n dígitos, ¿qué precisión tengo?

La misma cota permite estimar el error máximo cuando el formato ya fija la cantidad de dígitos disponibles. Si se dispone de n bits para la parte fraccionaria en base 2, el error queda en el orden de:

$$\text{error} < 2^{(-n)}$$

En el ejemplo del video se menciona IEEE 754 de precisión simple: la mantisa tiene 23 bits, pero si la parte entera del número ya consume varios bits, quedan menos bits disponibles para representar la fracción. Por eso, para números grandes, la parte fraccionaria se representa con menos detalle.

Idea de examen/trabajo: Mientras más grande sea la parte entera de un flotante, menos capacidad efectiva queda para conservar muchos dígitos fraccionarios dentro de la mantisa fija.

3.7. Redondeo en IEEE 754

Al final de la clase se aborda el redondeo. Como no todos los reales pueden representarse, el hardware debe escoger un número representable cercano. IEEE 754 ofrece varios modos:

Modo	Idea
Hacia el más próximo	El representable más cercano; es el redondeo usual.
Hacia cero	Truncamiento; elimina lo que no cabe sin alejarse del cero.
Hacia +infinito	Redondeo por exceso; el resultado queda hacia arriba.
Hacia -infinito	Redondeo por defecto; el resultado queda hacia abajo.

También se menciona la idea de bits de guarda: bits extra usados internamente para tomar una decisión de redondeo más correcta antes de guardar el resultado final en la mantisa disponible.

4. Cosas que conviene memorizar

- Sesgo del exponente: $\text{bias} = 2^{(k-1)} - 1$.
- Valor normalizado: $(-1)^s * (1.\text{fracción}) * 2^{(E-\text{bias})}$.
- Conversión fraccionaria a base b: multiplicar por b, tomar parte entera y repetir con la fracción.
- Error al cortar después de n dígitos: $|r - S_n| < b^{(-n)}$.
- Para precisión epsilon: $n > \log_b(1/\text{epsilon})$.
- 0.1 decimal no se representa exactamente en binario.
- Modos de redondeo: cercano, cero, +infinito, -infinito.
- En operaciones masivas de punto flotante, el error acumulado puede volverse significativo.

5. Posibles preguntas y respuestas rápidas

¿Por qué aparece error en flotantes? Porque la máquina usa una cantidad finita de bits y muchas fracciones tienen expansión infinita en base 2.

¿Qué hace el sesgo del exponente? Permite guardar exponentes positivos y negativos usando un campo binario sin signo.

¿Cuál es el algoritmo para convertir 0.r a binario? Multiplicar por 2, tomar la parte entera como bit, y repetir con la parte fraccionaria.

¿Cómo sé cuántos dígitos necesito para cierta precisión? Usando $b^{(-n)} < \epsilon$, o equivalentemente $n > \log_b(1/\epsilon)$.

¿Qué pasa si el resultado es demasiado pequeño? Puede ocurrir underflow; según el modo de manejo, se puede redondear, llevar a 0, usar subnormales o disparar excepción.

¿Qué diferencia hay entre truncar y redondear al más próximo? Truncar va hacia cero; redondear al más próximo escoge el representable más cercano.